Phase 4: Shot Detection, Frame Sampling and Quality Filtering

Complete source code (CineComic-MML)

What it does. Crops black bars, detects shots, samples frames (about 2 fps plus first, middle and last frame of each shot), removes dark, relatively blurry and near-duplicate frames, and builds 3-10 candidate frames for every dialogue segment from Phase 2.

Reads: data/raw/{id}.*, data/subtitles/{id}.json **Writes:** data/shots/{id}.json, data/shots/{id}.metrics.json, data/frames/{id}/shot_xxx/*.jpg, results/phase4/{id}_contact.jpg

Install and run

```
pip install -r requirements.txt
python -m src.video.phase4 --config configs/phase4.yaml --video clip_001
# with ground truth for the gate:
python -m src.video.phase4 --video clip_001 --gt-boundaries cuts.json --gt-crop 0,140,1920,940
pytest tests/test_phase4.py
```

Gate (suggested starting values, set your own after the first run)

Boundary F1 >= 0.85; segment coverage 100%; no candidate outside its shot; black-border rate <= 2%; crop IoU >= 0.95.

Files in this document

#	File	Purpose
1	`requirements.txt`	Python dependencies.
2	`configs/phase4.yaml`	All thresholds and settings for Phase 4 (nothing is hard-coded).
3	`src/video/black_bar.py`	Black-bar (letterbox) detection; returns the crop box in original pixels.
4	`src/video/shot_detection.py`	Shot boundary detection with PySceneDetect.
5	`src/video/quality_filter.py`	Sharpness, brightness and pHash statistics; dark, blur and duplicate filters.
6	`src/video/frame_extraction.py`	Sampling plan and the single sequential decode pass that saves cropped frames.
7	`src/evaluation/shot_metrics.py`	Boundary precision/recall/F1 and crop-box IoU.
8	`src/video/phase4.py`	The Phase 4 stage: run, candidates, evaluate, gate, contact sheet, CLI.
9	`tests/test_phase4.py`	Unit and end-to-end tests on a synthetic video.

Folder layout: configs/ for YAML, src/video/ and src/evaluation/ for code (each with an empty __init__.py, plus src/__init__.py), tests/ for pytest. Create the three empty __init__.py files before running.

1. requirements.txt

Python dependencies.

```
opencv-python>=4.8
scenedetect[opencv]>=0.6.2
imagehash>=4.3
Pillow
numpy
pyyaml
pytest
```

2. configs/phase4.yaml

All thresholds and settings for Phase 4 (nothing is hard-coded).

```
data_dir: data
results_dir: results
seed: 42

phase4:
  black_bar:
    n_samples: 30          # frames sampled across the video
    thresh: 16.0           # row/col mean intensity below this = black bar
    min_keep_frac: 0.5     # if crop keeps less than this area, fall back to full frame
  shots:
    detector: content      # content | adaptive
    threshold: 27.0        # ContentDetector threshold (adaptive: adaptive_threshold)
    min_scene_len_s: 0.5
  sampling:
    fps: 2.0               # regular grid, plus first/middle/last frame of each shot
    window_pad_s: 0.5      # dialogue window = [start - pad, end + pad] within its shot
    max_candidates_per_segment: 10
    min_per_segment: 3
    jpeg_quality: 90
  quality:
    min_brightness: 8.0    # rejects black frames from fades
    blur_rel_thresh: 0.4   # blurry if sharpness < this * video-median sharpness (relative, not absolute)
    phash_hamming: 5       # near-duplicate if Hamming distance <= this (within a shot)
    border_dark_thresh: 16.0
  contact_sheet: {n_shots: 20, cols: 5, thumb_w: 256}
  gate:                    # suggested starting values; justify your own in the report
    boundary_f1: 0.85
    boundary_tol_s: 0.5
    coverage: 1.0
    crop_iou: 0.95
    max_black_border_rate: 0.02
```

3. src/video/black_bar.py

Black-bar (letterbox) detection; returns the crop box in original pixels.

```
"""Black-bar (letterbox/pillarbox) detection. Returns crop box [x0, y0, x1, y1] in ORIGINAL pixels."""
from __future__ import annotations
import cv2
import numpy as np


def crop_from_gray_frames(grays: list[np.ndarray], thresh: float, min_keep_frac: float) -> list[int]:
    H, W = grays[0].shape
    full = [0, 0, W, H]
    # A row is "bar" only if it stays dark in EVERY sampled frame (max over frames of the row mean).
    row_max = np.max([g.mean(axis=1) for g in grays], axis=0)
    rows = np.where(row_max > thresh)[0]
    if len(rows) == 0:
        return full
    y0, y1 = int(rows[0]), int(rows[-1]) + 1
    # Column profile measured only inside the content rows, so horizontal bars don't dilute it.
    col_max = np.max([g[y0:y1].mean(axis=0) for g in grays], axis=0)
    cols = np.where(col_max > thresh)[0]
    if len(cols) == 0:
        return full
    x0, x1 = int(cols[0]), int(cols[-1]) + 1
    if (x1 - x0) * (y1 - y0) < min_keep_frac * W * H:
        return full
    return [x0, y0, x1, y1]


def detect_black_bars(video_path, n_samples=30, thresh=16.0, min_keep_frac=0.5) -> list[int]:
    cap = cv2.VideoCapture(str(video_path))
```

```
    n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    W = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)); H = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    idxs = (np.linspace(0, n - 1, n_samples + 2)[1:-1] if n > n_samples else np.arange(n)).astype(int)
    grays = []
    for i in idxs:
        cap.set(cv2.CAP_PROP_POS_FRAMES, int(i))
        ok, f = cap.read()
        if ok:
            grays.append(cv2.cvtColor(f, cv2.COLOR_BGR2GRAY))
    cap.release()
    if not grays:
        return [0, 0, W, H]
    return crop_from_gray_frames(grays, thresh, min_keep_frac)
```

4. src/video/shot_detection.py

Shot boundary detection with PySceneDetect.

```
"""Shot boundary detection with PySceneDetect. Returns [{'shot_id', 'start', 'end'}] in seconds."""
from __future__ import annotations
from scenedetect import detect, ContentDetector, AdaptiveDetector


def detect_shots(video_path, duration_s: float, fps: float, detector="content",
                 threshold=27.0, min_scene_len_s=0.5) -> list[dict]:
    min_len = max(1, int(round(min_scene_len_s * fps)))
    if detector == "adaptive":
        det = AdaptiveDetector(adaptive_threshold=threshold, min_scene_len=min_len)
    else:
        det = ContentDetector(threshold=threshold, min_scene_len=min_len)
    scenes = detect(str(video_path), det)
    if not scenes:  # no cut found: the whole video is one shot
        return [{"shot_id": 0, "start": 0.0, "end": float(duration_s)}]
    shots = [{"shot_id": k, "start": s.get_seconds(), "end": e.get_seconds()}
             for k, (s, e) in enumerate(scenes)]
    shots[0]["start"] = 0.0
    shots[-1]["end"] = float(duration_s)
    return shots
```

5. src/video/quality_filter.py

Sharpness, brightness and pHash statistics; dark, blur and duplicate filters.

```
"""Per-frame quality statistics and the relative blur / duplicate / dark filters."""
from __future__ import annotations
from collections import defaultdict
import cv2
import imagehash
import numpy as np
from PIL import Image


def frame_stats(crop_bgr: np.ndarray, border_thresh: float = 16.0) -> dict:
    gray = cv2.cvtColor(crop_bgr, cv2.COLOR_BGR2GRAY)
    ph = imagehash.phash(Image.fromarray(cv2.cvtColor(crop_bgr, cv2.COLOR_BGR2RGB)))
    return {
        "sharp": float(cv2.Laplacian(gray, cv2.CV_64F).var()),
        "brightness": float(gray.mean()),
        "contrast": float(gray.std()),
        "phash": str(ph),
        "border_dark": has_black_bars(gray, border_thresh),
    }


def has_black_bars(gray: np.ndarray, thresh: float = 16.0) -> bool:
    return bool(gray[0].mean() < thresh or gray[-1].mean() < thresh
                or gray[:, 0].mean() < thresh or gray[:, -1].mean() < thresh)


def hamming_hex(a: str, b: str) -> int:
    return bin(int(a, 16) ^ int(b, 16)).count("1")


def apply_filters(frames: list[dict], q: dict) -> None:
    """Sets rec['reject'] to None | 'dark' | 'blurry' | 'duplicate' (in place)."""
    for f in frames:
        f["reject"] = "dark" if f["brightness"] < q["min_brightness"] else None
    live = [f for f in frames if f["reject"] is None]
    if not live:
        return
```

```
    median = float(np.median([f["sharp"] for f in live]))
    by_shot = defaultdict(list)
    for f in live:
        by_shot[f["shot_id"]].append(f)
    for fs in by_shot.values():
        sharpest = max(fs, key=lambda f: f["sharp"])  # a shot never loses its best frame to blur
        for f in fs:
            if f is not sharpest and f["sharp"] < q["blur_rel_thresh"] * median:
                f["reject"] = "blurry"
    # Near-duplicates inside a shot: sharpest of each group survives (deterministic tie-break on t)
    for fs in by_shot.values():
        kept: list[dict] = []
        for f in sorted((x for x in fs if x["reject"] is None), key=lambda x: (-x["sharp"], x["t"])):
            if any(hamming_hex(f["phash"], k["phash"]) <= q["phash_hamming"] for k in kept):
                f["reject"] = "duplicate"
            else:
                kept.append(f)
```

6. src/video/frame_extraction.py

Sampling plan and the single sequential decode pass that saves cropped frames.

```
"""Sampling plan (which frame indices to decode) and one sequential decode pass."""
from __future__ import annotations
import logging
from pathlib import Path
import cv2
import numpy as np
from src.video.quality_filter import frame_stats

log = logging.getLogger("phase4")


def add_index_ranges(shots: list[dict], fps: float, n_frames: int) -> list[dict]:
    """Give every shot an inclusive frame range [i0, i1]; shots tile the video with no overlap."""
    for k, s in enumerate(shots):
        s["i0"] = min(int(round(s["start"] * fps)), n_frames - 1)
        nxt = int(round(shots[k + 1]["start"] * fps)) if k + 1 < len(shots) else n_frames
        s["i1"] = max(s["i0"], min(nxt - 1, n_frames - 1))
        s["start"], s["end"] = s["i0"] / fps, (s["i1"] + 1) / fps
    return shots


def assign_shot(seg: dict, shots: list[dict]) -> dict:
    """Shot with the largest time overlap with the segment (nearest shot if no overlap)."""
    ov = [max(0.0, min(seg["end"], s["end"]) - max(seg["start"], s["start"])) for s in shots]
    if max(ov) > 0:
        return shots[int(np.argmax(ov))]
    mid = (seg["start"] + seg["end"]) / 2
    return min(shots, key=lambda s: 0 if s["start"] <= mid < s["end"] else min(abs(mid - s["start"]), abs(
          mid - s["end"])))


def segment_window(seg: dict, shot: dict, pad: float) -> tuple[float, float]:
    lo, hi = max(shot["start"], seg["start"] - pad), min(shot["end"], seg["end"] + pad)
    if hi < lo:
        lo = hi = min(max((seg["start"] + seg["end"]) / 2, shot["start"]), shot["end"])
    return lo, hi


def build_sampling_plan(shots, segments, fps, cfg) -> dict[int, tuple[int, str]]:
    """Returns {frame_idx: (shot_id, frame_id)}. frame_ids are numbered by time inside each shot."""
    step = max(1, int(round(fps / cfg["fps"])))
    per_shot = {}
    for s in shots:
        i0, i1 = s["i0"], s["i1"]
        per_shot[s["shot_id"]] = set(range(i0, i1 + 1, step)) | {i0, (i0 + i1) // 2, i1}
    for seg in segments:
        s = assign_shot(seg, shots)
        lo_t, hi_t = segment_window(seg, s, cfg["window_pad_s"])
        lo = min(max(int(round(lo_t * fps)), s["i0"]), s["i1"])
        hi = min(max(int(round(hi_t * fps)), lo), s["i1"])
        have = [i for i in per_shot[s["shot_id"]] if lo <= i <= hi]
        if len(have) < cfg["min_per_segment"]:  # short window: add evenly spaced extra frames
            extra = np.linspace(lo, hi, cfg["min_per_segment"]).round().astype(int)
            per_shot[s["shot_id"]] |= {int(e) for e in extra}
    plan = {}
    for sid, idxs in per_shot.items():
        for n, i in enumerate(sorted(idxs)):
            plan[i] = (sid, f"s{sid:02d}_f{n:03d}")
```

```
    return plan


def extract_frames(video_path, plan, crop_box, frames_dir: Path, fps: float,
                   jpeg_quality=90, border_thresh=16.0) -> list[dict]:
    """One sequential pass (exact frames, no seeking). Saves cropped JPEGs and computes quality stats."""
    x0, y0, x1, y1 = crop_box
    cap = cv2.VideoCapture(str(video_path))
    last, idx, records = max(plan), 0, []
    while idx <= last:
        if not cap.grab():
            log.warning("decode ended early at frame %d (expected up to %d)", idx, last)
            break
        if idx in plan:
            ok, frame = cap.retrieve()
            if ok:
                sid, fid = plan[idx]
                crop = frame[y0:y1, x0:x1]
                path = frames_dir / f"shot_{sid:03d}" / f"{fid}.jpg"
                path.parent.mkdir(parents=True, exist_ok=True)
                cv2.imwrite(str(path), crop, [cv2.IMWRITE_JPEG_QUALITY, jpeg_quality])
                rec = {"frame_id": fid, "shot_id": sid, "idx": idx, "t": idx / fps, "path": path.as_posix(
                      )}
                rec.update(frame_stats(crop, border_thresh))
                records.append(rec)
        idx += 1
    cap.release()
    return records
```

7. src/evaluation/shot_metrics.py

Boundary precision/recall/F1 and crop-box IoU.

```
from __future__ import annotations


def boundary_prf(pred: list[float], gt: list[float], tol: float = 0.5) -> dict:
    """One-to-one greedy matching of cut times within +/- tol seconds."""
    pred, gt = sorted(pred), sorted(gt)
    used, tp = set(), 0
    for p in pred:
        cands = [(abs(p - g), j) for j, g in enumerate(gt) if j not in used and abs(p - g) <= tol]
        if cands:
            used.add(min(cands)[1]); tp += 1
    P = tp / len(pred) if pred else (1.0 if not gt else 0.0)
    R = tp / len(gt) if gt else (1.0 if not pred else 0.0)
    F = 2 * P * R / (P + R) if P + R else 0.0
    return {"boundary_precision": P, "boundary_recall": R, "boundary_f1": F, "tp": tp}


def box_iou(a, b) -> float:
    ix = max(0, min(a[2], b[2]) - max(a[0], b[0])); iy = max(0, min(a[3], b[3]) - max(a[1], b[1]))
    inter = ix * iy
    union = (a[2] - a[0]) * (a[3] - a[1]) + (b[2] - b[0]) * (b[3] - b[1]) - inter
    return inter / union if union else 0.0
```

8. src/video/phase4.py

The Phase 4 stage: run, candidates, evaluate, gate, contact sheet, CLI.

```
"""Phase 4: shot detection, frame sampling, quality filtering, per-segment candidate frames.

Reads : data/raw/{id}.*, data/subtitles/{id}.json (Phase 2; optional for standalone runs)
Writes: data/shots/{id}.json (+ .metrics.json), data/frames/{id}/shot_xxx/*.jpg

Run:  python -m src.video.phase4 --config configs/phase4.yaml --video clip_001
"""
from __future__ import annotations
import argparse, hashlib, json, logging, random, time
from collections import defaultdict
from pathlib import Path

import cv2
import numpy as np
import yaml

from src.evaluation.shot_metrics import boundary_prf, box_iou
from src.video.black_bar import detect_black_bars
from src.video.frame_extraction import (add_index_ranges, assign_shot, build_sampling_plan,
                                        extract_frames, segment_window)
```

```
from src.video.quality_filter import apply_filters
from src.video.shot_detection import detect_shots

log = logging.getLogger("phase4")


def build_segment_candidates(segments, shots, frames, cfg) -> tuple[dict, int]:
    """segment_id -> [frame_id]. Guarantees min_per_segment where the shot has enough usable frames."""
    by_shot = defaultdict(list)
    for f in frames:
        by_shot[f["shot_id"]].append(f)
    mx, mn, pad = cfg["max_candidates_per_segment"], cfg["min_per_segment"], cfg["window_pad_s"]
    out, backfilled = {}, 0
    for seg in segments:
        shot = assign_shot(seg, shots)
        lo, hi = segment_window(seg, shot, pad)
        pool = by_shot[shot["shot_id"]]
        chosen = [f for f in pool if f["reject"] is None and lo <= f["t"] <= hi]
        if len(chosen) > mx:  # cap, evenly spaced in time
            sel = sorted(set(np.linspace(0, len(chosen) - 1, mx).round().astype(int)))
            chosen = [chosen[i] for i in sel]
        if len(chosen) < mn:  # backfill with the sharpest blurry/duplicate frames from the window
            rej = sorted((f for f in pool if f["reject"] in ("blurry", "duplicate") and lo <= f["t"] <= hi
                  ),
                         key=lambda f: (-f["sharp"], f["t"]))
            for f in rej[: mn - len(chosen)]:
                f["reject"], f["backfilled"] = None, True
                chosen.append(f); backfilled += 1
        if len(chosen) < mn:  # still short: nearest kept frames of the same shot
            mid = (seg["start"] + seg["end"]) / 2
            ids = {f["frame_id"] for f in chosen}
            near = sorted((f for f in pool if f["reject"] is None and f["frame_id"] not in ids),
                          key=lambda f: abs(f["t"] - mid))
            chosen += near[: mn - len(chosen)]
        out[str(seg["segment_id"])] = [f["frame_id"] for f in sorted(chosen, key=lambda f: f["t"])]
    return out, backfilled


class Phase4Stage:
    name = "shots"
    requires = ["subtitles"]

    def __init__(self, cfg: dict, video_id: str):
        self.cfg, self.p4, self.video_id = cfg, cfg["phase4"], video_id
        d = Path(cfg["data_dir"])
        found = sorted((d / "raw").glob(f"{video_id}.*"))
        if not found:
            raise FileNotFoundError(f"no video data/raw/{video_id}.*")
        self.video_path = found[0]
        self.out = d / "shots" / f"{video_id}.json"
        self.metrics_path = self.out.with_suffix(".metrics.json")
        self.frames_dir = d / "frames" / video_id
        self.transcript_path = d / "subtitles" / f"{video_id}.json"
        self.results_dir = Path(cfg.get("results_dir", "results")) / "phase4"

    # ---------- helpers ----------
    def cache_key(self) -> str:
        blob = json.dumps({"p4": self.p4, "seed": self.cfg.get("seed")}, sort_keys=True)
        return hashlib.sha1(blob.encode()).hexdigest()[:8]

    def _segments(self) -> list[dict]:
        if not self.transcript_path.exists():
            log.warning("no transcript at %s: segment_candidates will be empty", self.transcript_path)
            return []
        segs = json.loads(self.transcript_path.read_text(encoding="utf-8"))["segments"]
        return sorted(segs, key=lambda s: s["start"])

    # ---------- main work ----------
    def run(self, force=False) -> None:
        if self.out.exists() and not force:
            if json.loads(self.out.read_text()).get("config_hash") == self.cache_key():
                log.info("cache hit for %s, skipping", self.video_id); return
        random.seed(self.cfg.get("seed", 42)); np.random.seed(self.cfg.get("seed", 42))
        cap = cv2.VideoCapture(str(self.video_path))
        fps = cap.get(cv2.CAP_PROP_FPS); n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
        cap.release()
        duration = n_frames / fps
        p = self.p4

        crop_box = detect_black_bars(self.video_path, **p["black_bar"])
```

```
        log.info("crop_box=%s", crop_box)
        shots = detect_shots(self.video_path, duration, fps, p["shots"]["detector"],
                             p["shots"]["threshold"], p["shots"]["min_scene_len_s"])
        add_index_ranges(shots, fps, n_frames)
        log.info("%d shots", len(shots))

        segments = self._segments()
        plan = build_sampling_plan(shots, segments, fps, p["sampling"])
        log.info("decoding %d sampled frames", len(plan))
        if self.frames_dir.exists():  # stale frames from an earlier config must not survive
            for old in self.frames_dir.rglob("*.jpg"):
                old.unlink()
        frames = extract_frames(self.video_path, plan, crop_box, self.frames_dir, fps,
                                p["sampling"]["jpeg_quality"], p["quality"]["border_dark_thresh"])
        apply_filters(frames, p["quality"])
        cands, backfilled = build_segment_candidates(segments, shots, frames, p["sampling"])

        n_removed = defaultdict(int)
        for f in frames:
            if f["reject"] is not None:
                n_removed[f["reject"]] += 1
                Path(f["path"]).unlink(missing_ok=True)
        kept = [f for f in frames if f["reject"] is None]
        by_shot = defaultdict(list)
        for f in sorted(kept, key=lambda f: f["t"]):
            rec = {k: (round(f[k], 3) if isinstance(f[k], float) else f[k])
                   for k in ("frame_id", "idx", "t", "sharp", "brightness", "phash", "path", "border_dark"
                             )}
            if f.get("backfilled"):
                rec["backfilled"] = True
            by_shot[f["shot_id"]].append(rec)

        doc = {
            "video_id": self.video_id, "fps": fps, "n_frames": n_frames, "duration_s": duration,
            "crop_box": crop_box, "config_hash": self.cache_key(),
            "shots": [{"shot_id": s["shot_id"], "start": round(s["start"], 4), "end": round(s["end"], 4),
                       "frames": by_shot[s["shot_id"]]} for s in shots],
            "segment_candidates": cands,
            "stats": {"sampled": len(frames), "kept": len(kept), "removed": dict(n_removed),
                      "backfilled": backfilled, "n_segments": len(segments)},
        }
        self.out.parent.mkdir(parents=True, exist_ok=True)
        self.out.write_text(json.dumps(doc, indent=1), encoding="utf-8")
        self.make_contact_sheet(doc)

    # ---------- evaluation ----------
    def evaluate(self, gt: dict | None = None) -> dict:
        doc = json.loads(self.out.read_text())
        g = self.p4["gate"]; mn = self.p4["sampling"]["min_per_segment"]
        frames = {f["frame_id"]: (s, f) for s in doc["shots"] for f in s["frames"]}
        kept = [f for s in doc["shots"] for f in s["frames"]]
        cands = doc["segment_candidates"]
        m = {
            "n_shots": len(doc["shots"]), "n_frames_kept": len(kept), **{f"stats_{k}": v for k, v in doc["
                  stats"].items() if k != "removed"},
            "removed": doc["stats"]["removed"],
            "pct_removed": 1 - doc["stats"]["kept"] / max(1, doc["stats"]["sampled"]),
            "pool_per_min": len(kept) / (doc["duration_s"] / 60),
            "black_border_rate": sum(f["border_dark"] for f in kept) / max(1, len(kept)),
        }
        if cands:
            m["coverage"] = sum(len(v) >= mn for v in cands.values()) / len(cands)
            # unit-test style invariant: every candidate lies inside the shot it belongs to
            bad = 0
            for ids in cands.values():
                for fid in ids:
                    s, f = frames[fid]
                    bad += not (s["start"] - 1e-6 <= f["t"] <= s["end"] + 1e-6)
            m["candidates_outside_shot"] = bad
        if gt:
            if "boundaries" in gt:
                pred = [s["start"] for s in doc["shots"][1:]]
                m.update(boundary_prf(pred, gt["boundaries"], g["boundary_tol_s"]))
            if "crop_box" in gt:
                m["crop_iou"] = box_iou(doc["crop_box"], gt["crop_box"])
        return m

    def gate(self, m: dict) -> tuple[bool, str]:
        g, fails = self.p4["gate"], []
        if "coverage" in m and m["coverage"] < g["coverage"]: fails.append(f"coverage {m['coverage']:.2f}"
```

```
            )
        if m.get("candidates_outside_shot", 0): fails.append("candidates outside their shot")
        if m["black_border_rate"] > g["max_black_border_rate"]: fails.append(f"black borders {m['black_bor
            der_rate']:.3f}")
        if "boundary_f1" in m and m["boundary_f1"] < g["boundary_f1"]: fails.append(f"boundary F1 {m['boun
            dary_f1']:.2f}")
        if "crop_iou" in m and m["crop_iou"] < g["crop_iou"]: fails.append(f"crop IoU {m['crop_iou']:.2f}"
            )
        return (not fails, "; ".join(fails) or "all gates passed")

    def execute(self, gt=None, force=False) -> bool:
        t0 = time.time()
        self.run(force=force)
        m = self.evaluate(gt)
        ok, why = self.gate(m)
        m.update(runtime_s=time.time() - t0, passed=ok, reason=why, cfg=self.cache_key())
        self.metrics_path.write_text(json.dumps(m, indent=2))
        log.info("metrics: %s", json.dumps(m))
        return ok

    # ---------- manual visual check ----------
    def make_contact_sheet(self, doc: dict) -> None:
        cs = self.p4["contact_sheet"]
        shots = [s for s in doc["shots"] if s["frames"]]
        if not shots:
            return
        rnd = random.Random(self.cfg.get("seed", 42))
        pick = sorted(rnd.sample(shots, min(cs["n_shots"], len(shots))), key=lambda s: s["shot_id"])
        tw, tiles = cs["thumb_w"], []
        for s in pick:
            img = cv2.imread(s["frames"][len(s["frames"]) // 2]["path"])
            th = int(img.shape[0] * tw / img.shape[1])
            img = cv2.resize(img, (tw, th))
            cv2.putText(img, f"shot {s['shot_id']}", (6, 18), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255)
                , 2)
            tiles.append(cv2.resize(img, (tw, int(tw * 9 / 16))))
        while len(tiles) % cs["cols"]:
            tiles.append(np.zeros_like(tiles[0]))
        rows = [np.hstack(tiles[i:i + cs["cols"]]) for i in range(0, len(tiles), cs["cols"])]
        self.results_dir.mkdir(parents=True, exist_ok=True)
        cv2.imwrite(str(self.results_dir / f"{self.video_id}_contact.jpg"), np.vstack(rows))


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--config", default="configs/phase4.yaml")
    ap.add_argument("--video", required=True)
    ap.add_argument("--force", action="store_true")
    ap.add_argument("--gt-boundaries", help="JSON list of manual cut times in seconds")
    ap.add_argument("--gt-crop", help="manual crop box 'x0,y0,x1,y1'")
    a = ap.parse_args()
    cfg = yaml.safe_load(open(a.config))
    Path("logs").mkdir(exist_ok=True)
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s",
                        handlers=[logging.FileHandler(f"logs/{a.video}.log"), logging.StreamHandler()])
    gt = {}
    if a.gt_boundaries: gt["boundaries"] = json.load(open(a.gt_boundaries))
    if a.gt_crop: gt["crop_box"] = [int(x) for x in a.gt_crop.split(",")]
    ok = Phase4Stage(cfg, a.video).execute(gt or None, force=a.force)
    raise SystemExit(0 if ok else 1)


if __name__ == "__main__":
    main()
```

9. tests/test_phase4.py

Unit and end-to-end tests on a synthetic video.

```
import json
from pathlib import Path
import cv2
import numpy as np
import pytest
import yaml

pytest.importorskip("scenedetect")
pytest.importorskip("imagehash")
from src.evaluation.shot_metrics import boundary_prf, box_iou
from src.video.black_bar import crop_from_gray_frames
```

```
from src.video.frame_extraction import add_index_ranges, assign_shot
from src.video.phase4 import Phase4Stage
from src.video.quality_filter import apply_filters, hamming_hex

FPS, W, H, BAR = 25, 320, 240, 20


def _make_video(path: Path, shots=3, sec=2):
    vw = cv2.VideoWriter(str(path), cv2.VideoWriter_fourcc(*"mp4v"), FPS, (W, H))
    for k in range(shots):
        rs = np.random.RandomState(k)
        tex = cv2.resize(rs.randint(40, 255, (24, 40, 3), dtype=np.uint8), (W, H - 2 * BAR), interpolation
              =cv2.INTER_NEAREST)
        for i in range(FPS * sec):
            f = np.zeros((H, W, 3), np.uint8)
            f[BAR:H - BAR] = tex
            cv2.rectangle(f, (10 + i, BAR + 10), (40 + i, BAR + 40), (255, 255, 255), -1)  # slight motion
            vw.write(f)
    vw.release()


@pytest.fixture
def stage(tmp_path):
    cfg = yaml.safe_load(open(Path(__file__).parents[1] / "configs" / "phase4.yaml"))
    cfg["data_dir"], cfg["results_dir"] = str(tmp_path / "data"), str(tmp_path / "results")
    (tmp_path / "data" / "raw").mkdir(parents=True); (tmp_path / "data" / "subtitles").mkdir(parents=True)
    _make_video(tmp_path / "data" / "raw" / "t.mp4")
    segs = [{"segment_id": 1, "start": 0.2, "end": 1.0}, {"segment_id": 2, "start": 2.3, "end": 3.1},
            {"segment_id": 3, "start": 4.5, "end": 5.5}]
    (tmp_path / "data" / "subtitles" / "t.json").write_text(json.dumps({"segments": segs}))
    return Phase4Stage(cfg, "t")


def test_black_bar_crop():
    g = np.zeros((H, W), np.uint8); g[BAR:H - BAR] = 120
    assert crop_from_gray_frames([g, g], 16.0, 0.5) == [0, BAR, W, H - BAR]


def test_boundary_and_iou():
    assert boundary_prf([2.0, 4.1], [2.0, 4.0, 9.0], 0.5)["tp"] == 2
    assert box_iou([0, 0, 10, 10], [0, 0, 10, 10]) == 1.0


def test_dark_and_duplicate_filters():
    q = {"min_brightness": 8, "blur_rel_thresh": 0.4, "phash_hamming": 5}
    mk = lambda i, sharp, br, ph: dict(frame_id=i, shot_id=0, t=float(i), sharp=sharp, brightness=br, phas
          h=ph)
    fr = [mk(0, 100, 2, "0" * 16), mk(1, 100, 90, "0" * 16), mk(2, 120, 90, "0" * 15 + "1"), mk(3, 100, 90
          , "f" * 16)]
    apply_filters(fr, q)
    assert fr[0]["reject"] == "dark" and fr[1]["reject"] == "duplicate" and fr[2]["reject"] is None and fr
          [3]["reject"] is None
    assert hamming_hex("0" * 16, "f" * 16) == 64


def test_pipeline_end_to_end_and_determinism(stage):
    assert stage.execute(gt={"boundaries": [2.0, 4.0], "crop_box": [0, BAR, W, H - BAR]}, force=True)
    doc1 = json.loads(stage.out.read_text())
    m = json.loads(stage.metrics_path.read_text())
    assert doc1["crop_box"] == [0, BAR, W, H - BAR]
    assert len(doc1["shots"]) == 3 and m["boundary_f1"] == 1.0
    assert m["coverage"] == 1.0 and m["candidates_outside_shot"] == 0 and m["black_border_rate"] == 0
    shot_of = {f["frame_id"]: s for s in doc1["shots"] for f in s["frames"]}
    for sid, ids in doc1["segment_candidates"].items():  # all candidates inside the segment's shot
        assert len({shot_of[i]["shot_id"] for i in ids}) == 1
    for s in doc1["shots"]:  # saved frames have no bars
        img = cv2.imread(s["frames"][0]["path"]); assert img.shape[:2] == (H - 2 * BAR, W)
    stage.run(force=True)  # same input + config => same frame list
    doc2 = json.loads(stage.out.read_text())
    assert [f["frame_id"] for s in doc1["shots"] for f in s["frames"]] == [f["frame_id"] for s in doc2["sh
          ots"] for f in s["frames"]]
```